Jorge L. Rivera González

801-19-7676

May 1, 2025

CIIC4030-066

Programming Languages

Prof. Wilson Rivera

Assignment #4

For this assignment, a function interpreter() and it's helper function evaluate() was added to the code for Assignment #3. Once this modified A3_Jorge_Rivera.py file worked as expected for the test program Program_Test.txt of the previous submission, A3_Jorge_Rivera.py was refactored into the files expected of this submission: scanner.py, parser.py, interpreter.py, main.py.

The file scanner.py contains the lexical analyzer definition with it's list of tokens and regular expressions for this tokenization. It also contains the getter function get_lexer() that will be imported by main.py. The file parser.py contains the attribute grammar and operator precedence for the parser, and the get_parser() function that will be imported by main.py. The interpreter.py file contains the new additions to this assignment: the interpreter() function and the helper evaluate() function. The main.py file imports the getters for the scanner and parser, as well as the interpreter function. It functions the same as in the previous submission except that now lexer and parser are instantiated with the getters, and calls the interpreter() function to evaluate the generated AST. It was also changed to run tests faster from the terminal, but running the file itself still evaluates the previous test file.

The interpreter() function takes the generated AST in JSON format, assigns the definitions provided, and by calling the helper evaluate() function, evaluates the exec statement by treating statements as nodes in the tree. The helper function evaluate() recursively processes the AST nodes. Nodes for primitive values (types integer, floating point, string, boolean, or null) are evaluated to their value field. Nodes for identifier are evaluated by getting their value from the defined environment. Nodes for operations are checked for correct types of its values and if valid, evaluated. The professor said the dot operator could be ignored so I made it so if both values of the binary operation are integers, the operation returns a decimal number where the first value is the integer part, and the second value is the fractional part. If they are not both integers, then it would work as string concatenation. If-then-else blocks are evaluated by checking the condition and then deciding the branch to take. For let-in expressions, it creates a copy of the current

environment to operate inside the scope of the statement, processes the new variables and functions and then evaluates the environment.

Tests:

Test: Previous testing file:

A4_Jorge_Rivera > Program_Test.txt

```
1   // Use this code to test the scanner
2
3   func SomeFunction[n] :=
4       let
5           val r := 15 end
6       in
7           n*r
8       end
9   end
10
11  exec SomeFunction[3]
12
13  // Expected: 45
```

AST and result:

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"SomeFunction": {"type": "func", "name": "SomeFunction", "params": [{"type": "id", "id": "n"}], "stm": {"type":
"stm_let", "facts": {"r": {"type": "val", "name": "r", "stm": {"type": "stm_value", "type_value": "number", "value": 15}}},
"stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "n"}, "value2": {"type": "stm_id", "id": "r"}}}}},
"stm": {"type": "stm_func_call", "id_func": "SomeFunction", "args": [{"type": "stm_value", "type_value": "number", "value": 3}]}}

Evaluation result:  45

Finalizing Parsing
```

Test #1:

test1.txt

```
// Arithmetic operations and function calls

func Add[a, b] := a + b end
func Subtract[a, b] := a - b end
func Multiply[a, b] := a * b end
func Divide[a, b] := a / b end

exec Add[Multiply[5, 4], Divide[10, 2]]

// Expected: 25
```

AST and result: (I could not increase the size in my IDE without cropping the AST)

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"Add": {"type": "func", "name": "Add", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "+", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Subtract": {"type": "func", "name": "Subtract", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "-", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Multiply": {"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type""type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Divide": {"type": "func", "name": "Divide", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "/", "value1": {"ty"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", ""type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}"type": "func", "nam"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Divide": {"type": "func", "name": "Divide", "params": [{"type": "id", "id": "type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type":"type": "func", "name": "M"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Divide": {"type": "func", "name": "Divide", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "/", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}}, "stm": {"type": "stm_func_call", "id_func": "Add", "ar"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Divide": {"type": "func", "name": "Divide", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "/", "value1": {"type": "stm_id", "id": "a"type": "func", "name": "Multiply", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "*", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}, "Divide": {"type": "func", "name": "Divide", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_op", "op": "/", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}}, "stm": {"type": "stm_func_call", "id_func": "Add", "args": [{"type": "stm_func_call", "id_func": "Multiply", "args": [{"type": "stm_value", "type_value": "number", "value": 5}, {"type": "stm_value", "type_value": "number", "value": 4}]}, {"type": "stm_func_call", "id_func": "Divide", "args": [{"type": "stm_value", "type_value": "number", "value": 10}, {"type": "stm_value", "type_value": "number", "value": 2}]}]}}}

Evaluation result:  25.0

Finalizing Parsing
```

Test #2:

test2.txt

```
// Scope test with nested let expressions

val x := 10 end

func ScopeTest[y] :=
    let
        val x := 20 end
        val z := 5 end
    in
        let
            val x := 30 end
        in
            x + y + z
        end
    end
end

exec ScopeTest[x]

// Expected: 45
```

AST and result:

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"x": {"type": "val", "name": "x", "stm": {"type": "stm_value", "type_value": "number", "value": 10}}, "ScopeTest": {"type":
 "func", "name": "ScopeTest", "params": [{"type": "id", "id": "y"}], "stm": {"type": "stm_let", "facts": {"x": {"type": "val", "name":
 "x", "stm": {"type": "stm_value", "type_value": "number", "value": 20}}, "z": {"type": "val", "name": "z", "stm": {"type": "stm_value
", "type_value": "number", "value": 5}}}, "stm": {"type": "stm_let", "facts": {"x": {"type": "val", "name": "x", "stm": {"type": "stm_
value", "type_value": "number", "value": 30}}}, "stm": {"type": "stm_op", "op": "+", "value1": {"type": "stm_op", "op": "+", "value1":
 {"type": "stm_id", "id": "x"}, "value2": {"type": "stm_id", "id": "y"}}, "value2": {"type": "stm_id", "id": "z"}}}}}}, "stm": {"type"
: "stm_func_call", "id_func": "ScopeTest", "args": [{"type": "stm_id", "id": "x"}]}}

Evaluation result:  45

Finalizing Parsing
```

Test #3:

test3.txt

```
// Testing conditionals and logical operations

func Max[a, b] :=
    if a > b then
        a
    else
        b
    end
end

func LogicalTest[a, b] :=
    if a & b then
        1
    else
        if a | b then
            2
        else
            3
        end
    end
end

exec LogicalTest[true, false]

// Expected: 2
```

AST and result:

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"Max": {"type": "func", "name": "Max", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_if", "condition": {"type": "stm_op", "op": ">", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}, "then": {"type": "stm_id", "id": "a"}, "else": {"type": "stm_id", "id": "b"}}}, "LogicalTest": {"type": "func", "name": "LogicalTest", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}], "stm": {"type": "stm_if", "condition": {"type": "stm_op", "op": "&", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}, "then": {"type": "stm_value", "type_value": "number", "value": 1}, "else": {"type": "stm_if", "condition": {"type": "stm_op", "op": "|", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}, "then": {"type": "stm_value", "type_value": "number", "value": 2}, "else": {"type": "stm_value", "type_value": "number", "value": 3}}}}}, "stm": {"type": "stm_func_call", "id_func": "LogicalTest", "args": [{"type": "stm_value", "type_value": "boolean", "value": true}, {"type": "stm_value", "type_value": "boolean", "value": false}]}}

Evaluation result:  2

Finalizing Parsing
```

Test #4:

test4.txt

```
// Testing recursive function

func Factorial[n] :=
    if n = 0 then
        1
    else
        n * Factorial[n - 1]
    end
end

exec Factorial[5]

// Expected: 120
```

AST and result:

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"Factorial": {"type": "func", "name": "Factorial", "params": [{"type": "id", "id": "n"}], "stm": {"type": "stm_if", "condit
ion": {"type": "stm_op", "op": "=", "value1": {"type": "stm_id", "id": "n"}, "value2": {"type": "stm_value", "type_value": "number", "
value": 0}}, "then": {"type": "stm_value", "type_value": "number", "value": 1}, "else": {"type": "stm_op", "op": "*", "value1": {"type
": "stm_id", "id": "n"}, "value2": {"type": "stm_func_call", "id_func": "Factorial", "args": [{"type": "stm_op", "op": "-", "value1":
{"type": "stm_id", "id": "n"}, "value2": {"type": "stm_value", "type_value": "number", "value": 1}}]}}}}}}, "stm": {"type": "stm_func_c
all", "id_func": "Factorial", "args": [{"type": "stm_value", "type_value": "number", "value": 5}]}}

Evaluation result:  120

Finalizing Parsing
```

Test #5: **Inspired changes**

test5.txt

```
// String concatenation

func CombineStrings[a, b] :=
    a . b
end

exec CombineStrings["Hello", "World"]

// Expected: HelloWorld
```

AST and result: Pre correction

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"CombineStrings": {"type": "func", "name": "CombineStrings", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"
}], "stm": {"type": "stm_op", "op": ".", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}}, "stm": {
"type": "stm_func_call", "id_func": "CombineStrings", "args": [{"type": "stm_value", "type_value": "string", "value": "\"Hello\""}, {"
type": "stm_value", "type_value": "string", "value": "\"World\""}]}}

Evaluation result:  "Hello""World"

Finalizing Parsing
```

This test revealed I need to remove the double quotes from parsed strings.

AST and result: Post correction

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"CombineStrings": {"type": "func", "name": "CombineStrings", "params": [{"type": "id", "id": "a"}, {"type": "id", "id": "b"}]
, "stm": {"type": "stm_op", "op": ".", "value1": {"type": "stm_id", "id": "a"}, "value2": {"type": "stm_id", "id": "b"}}}}, "stm": {"typ
e": "stm_func_call", "id_func": "CombineStrings", "args": [{"type": "stm_value", "type_value": "string", "value": "Hello"}, {"type": "st
m_value", "type_value": "string", "value": "World"}]}}

Evaluation result:  HelloWorld

Finalizing Parsing
```

Test #6:

test6.txt

```
1   // Raise type error
2
3   func BadFunction[n] :=
4       n + "string"
5   end
6
7   exec BadFunction[10]
8
9   // Expected: TypeError
```

AST and result:

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"BadFunction": {"type": "func", "name": "BadFunction", "params": [{"type": "id", "id": "n"}], "stm": {"type": "stm_op", "op":
 "+", "value1": {"type": "stm_id", "id": "n"}, "value2": {"type": "stm_value", "type_value": "string", "value": "string"}}}}, "stm": {"t
ype": "stm_func_call", "id_func": "BadFunction", "args": [{"type": "stm_value", "type_value": "number", "value": 10}]}}

Execution error: TypeError: '+' requires both values to be numbers of type float or integer, types are: int and str.

Finalizing Parsing
```

Test #7:

test7.txt

```
1   // Goofy dot operator
2
3   func MakeDecimal[whole, fractional] :=
4       whole . fractional
5   end
6
7   exec MakeDecimal[3, 14]
8
9   // Expected: 3.14
```

AST and result:

```
Initiating Parsing

Abstract Syntax Tree:

{"facts": {"MakeDecimal": {"type": "func", "name": "MakeDecimal", "params": [{"type": "id", "id": "whole"}, {"type": "id", "id": "fracti
onal"}], "stm": {"type": "stm_op", "op": ".", "value1": {"type": "stm_id", "id": "whole"}, "value2": {"type": "stm_id", "id": "fractiona
l"}}}}, "stm": {"type": "stm_func_call", "id_func": "MakeDecimal", "args": [{"type": "stm_value", "type_value": "number", "value": 3}, {
"type": "stm_value", "type_value": "number", "value": 14}]}}

Evaluation result:  3.14

Finalizing Parsing
```